Project - Low Level Design

on

Autonomous Logistics Researcher Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	HATIM BOHRA	EN22CS301418
2	INDRA KAMAL PATEL	EN22CS301434
3	GOUTAM JAT	EN22CS301371
4	ISHIKA AGRAWAL	EN22CS301442
5	HARSH SOLANKI	EN22CS301394
6	HITESH AHUJA	EN22CS301426

Group Name: Group 09D4

Project Number: AAI – 45

Industry Mentor Name:

University Mentor Name:

Academic Year : 2025-26

Table of Contents

1. **Introduction.**
   1.1. Scope of the document.
   1.2. **Intended Audience**
   1.3. **System overview.**

2. **System Design.**
   2.1. Application Design

1. Introduction.

1.1. Scope of the document.

This Low Level Design (LLD) document provides granular technical specifications for every software component of the Autonomous Logistics Researcher Agent. It covers internal module design, agent configuration, task prompts, tool integration, data flow between components, and persistence strategy — at a level of detail sufficient for a developer to implement the system from scratch without additional design artefacts.

This document does not address deployment infrastructure, CI/CD pipelines, or business requirements (covered in HLD and BRD).

1.2. Intended Audience

- Backend developers implementing agent modules in Python 3.10+
- ML engineers configuring LLM prompts and CrewAI orchestration
- QA engineers designing test cases for self-correction behaviour

1.3. System overview.

The Autonomous Logistics Researcher Agent is a multi-agent AI system that automates deep-dive market research in the global logistics and supply chain domain. Given a complex research query, the system autonomously navigates the live web, extracts and cross-verifies data points from authoritative sources (BIMCO, Freightos, Lloyd's List, port authority bulletins), and synthesises findings into a structured, schema-compliant Markdown report persisted to a local knowledge repository for future RAG use.

Core capabilities: Autonomous web search · multi-source verification · self-correction on thin results (max_iter=5) · structured Markdown output · RAG-ready knowledge persistence via ChromaDB.

2. System Design.

2.1. Application Design

The application is structured as a layered Python package where each layer has a single responsibility and communicates through well-defined interfaces. The diagram below shows the complete system architect

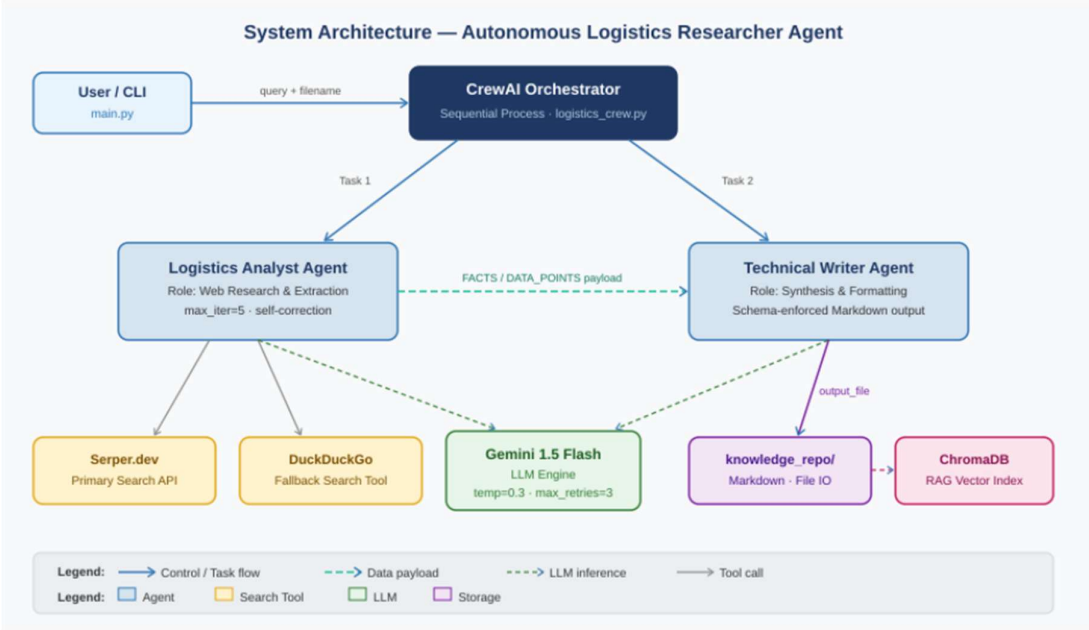


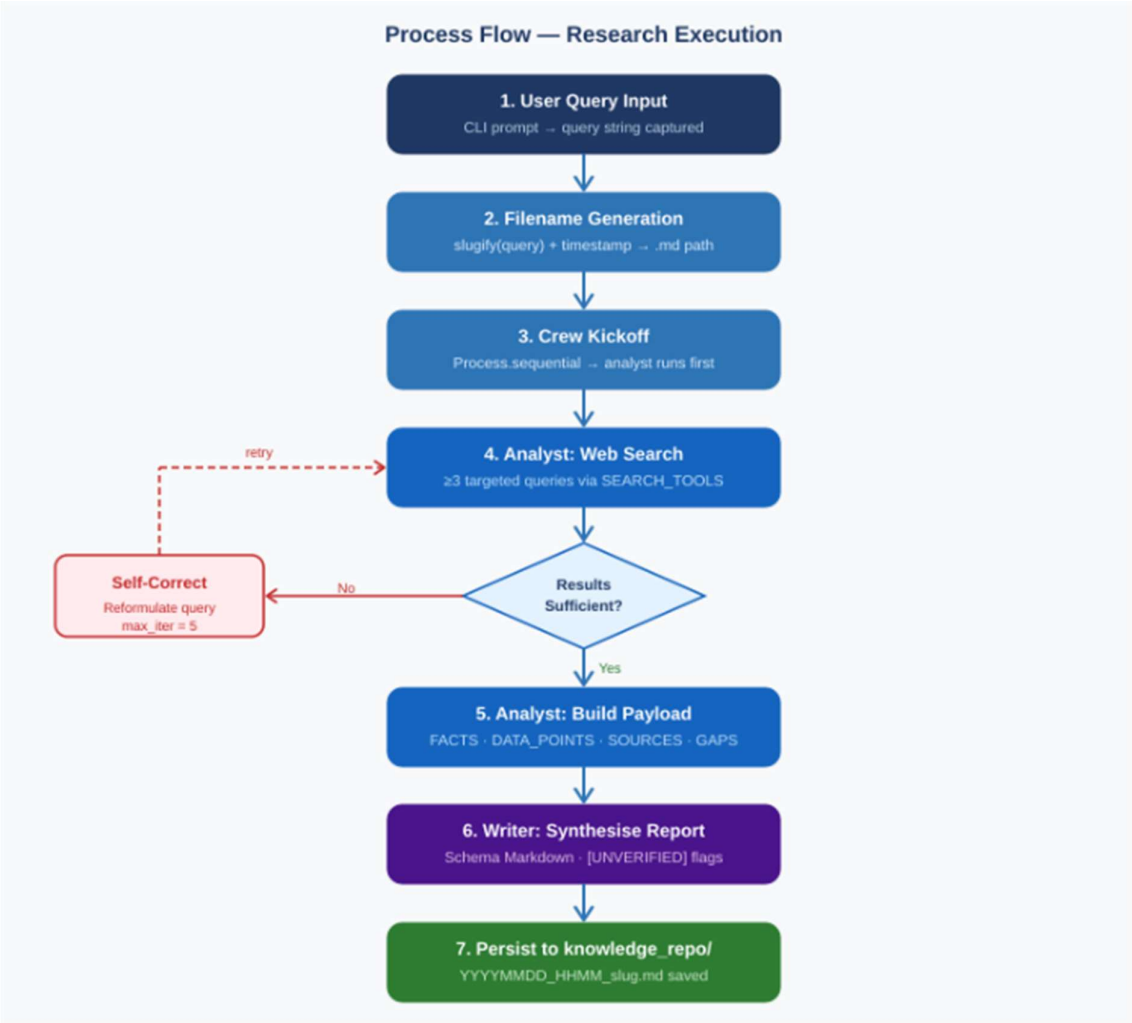
Figure 1 — System Architecture Diagram

2.2. Process Flow.

The diagram below illustrates the complete execution flow from user query input to persisted Markdown report, including the self-correction loop triggered when search results are insufficient.

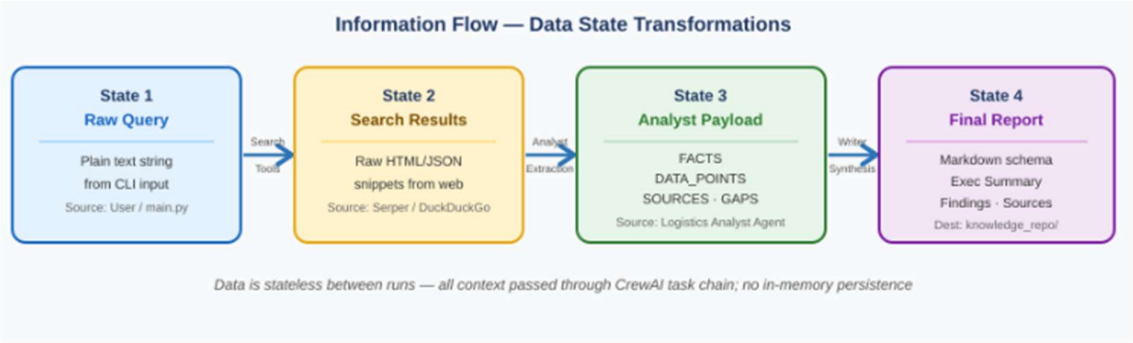
Step-by-Step Process

Step	Actor	Action
1	User / main.py	User enters a logistics research query via CLI prompt
2	main.py	slugify(query) + datetime.now() generates unique output filename
3	logistics_crew.py	Crew assembled with Process.sequential; crew.kickoff() invoked
4	Logistics Analyst	Runs ≥3 targeted web searches via SEARCH_TOOLS; extracts facts, metrics, URLs
5	Logistics Analyst	Self-correction: if results thin/stale, reformulates query and retries (max_iter=5)
6	CrewAI Orchestrator	Analyst output passed as context to Technical Writer task
7	Technical Writer	Synthesises FACTS/DATA_POINTS/SOURCES payload into Markdown report
8	CrewAI / File IO	Report saved to knowledge_repo/{filename}.md via output_file parameter



2.3. Information Flow.

Data undergoes four distinct state transformations as it moves through the system. No state is persisted in memory between runs — all context is carried in the CrewAI task chain and written to disk at completion.



2.4. Components Design

The component diagram below shows the internal structure of each module and its dependencies.

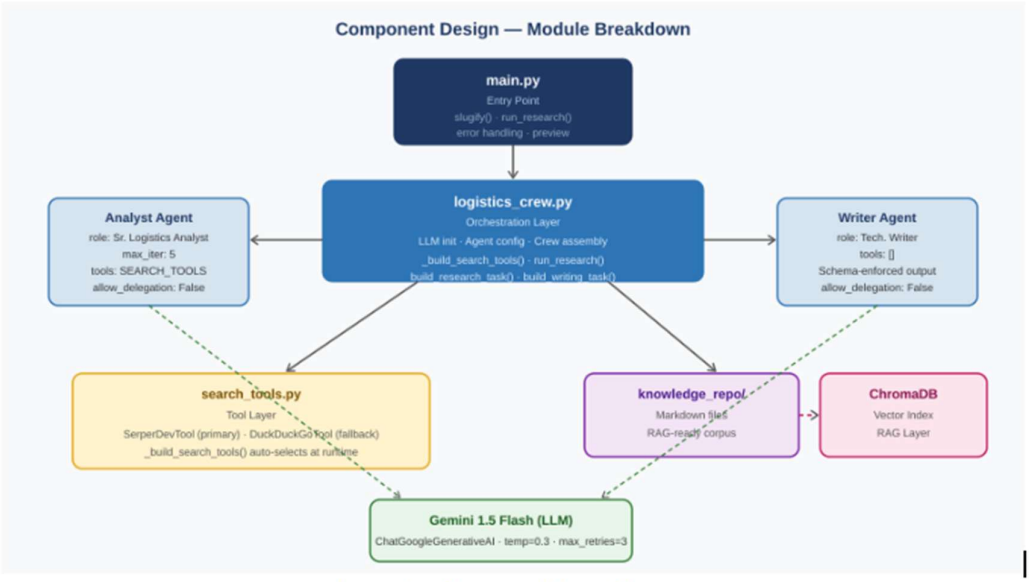


Figure 4 — Component Design Diagram

2.5. Key Design Considerations

Consideration	Design Decision & Rationale
Self-Correction	max_iter=5 on the Analyst. CrewAI re-invokes with previous output as context when LLM evaluates result as insufficient.
Tool Modularity	SEARCH_TOOLS assembled at startup by _build_search_tools(). Swapping to a premium API requires changing one return statement.
Prompt Isolation	All prompt text lives in Task description strings, not agent backstory. Agents are reusable; prompt tuning is localised.
Statelessness	No in-memory state between runs. All context carried via CrewAI task chain. Enables horizontal scalability.
API Key Safety	Keys loaded from .env via python-dotenv. Hard fail at import time if GOOGLE_API_KEY missing — no silent errors.
LLM Swappability	Single llm object injected into both agents. Switching to GPT-4o or Claude requires changing only the constructor.
Rate Limit	max_retries=3 + request_timeout=60 handle HTTP 429 / 503 with

Handling	LangChain exponential backoff decorator.
RAG Extensibility	knowledge_repo/ naming, Markdown schema, and section headers chosen to maximise chunk quality for MarkdownTextSplitter.

2.6 API Catalogue

All external APIs and internal Python interfaces consumed or exposed by the system are documented below.

External APIs

API	Endpoint	Auth	Used For
Google Gemini	generativelanguage.googleapis.com	GOOGLE_API_KEY	LLM inference for both agents
Serper.dev	google.serper.dev/search	SERPER_API_KEY	Primary web search (optional)
DuckDuckGo	duckduckgo.com (scrape)	None	Free fallback web search
Gemini Embeddings	models/embedding-001	GOOGLE_API_KEY	RAG vector indexing (Phase 5)

3. References

#	Resource	URL / Citation
1	CrewAI Documentation	https://docs.crewai.com

2	LangChain Google GenAI	https://python.langchain.com/docs/integrations/llms/google_ai
3	Google Gemini API Docs	https://ai.google.dev/docs
4	Serper.dev API	https://serper.dev/docs
5	DuckDuckGo Search (PyPI)	https://pypi.org/project/duckduckgo-search
6	ChromaDB Documentation	https://docs.trychroma.com
7	LangChain Chroma Integration	https://python.langchain.com/docs/integrations/vectorstores/chroma
8	python-dotenv	https://pypi.org/project/python-dotenv
9	CrewAI BaseTool Reference	https://docs.crewai.com/concepts/tools
10	Gemini Embeddings Guide	https://ai.google.dev/docs/embeddings_guide